

JAVA BANK ACCOUNT SYSTEM

1. AIM

To design and implement a Bank Account System in Java using object-oriented programming concepts such as classes, objects, attributes, methods, constructors, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.

2. DESCRIPTION

The Bank Account System contains a base class named Account. It stores account number, account holder name, balance, and account type. The Account class provides deposit(), withdraw(), transfer(), and displayAccountDetails() methods.

SavingsAccount extends Account and adds interestRate with calculateInterest(). CurrentAccount also extends Account and adds overdraftLimit. It overrides withdraw() to allow withdrawal up to the permitted overdraft limit. Objects of both account types are created and transactions are performed.

3. ALGORITHM

1. Start the program.
2. Create the Account base class.
3. Declare account number, holder name, balance, and account type.
4. Create a constructor to initialize account details.
5. Implement deposit() to add money to the balance.
6. Implement withdraw() to withdraw money when sufficient balance is available.
7. Implement transfer() to move money from one account to another.
8. Implement displayAccountDetails() to show account information.
9. Create SavingsAccount by inheriting Account and add interestRate.
10. Implement calculateInterest() for the savings account.
11. Create CurrentAccount by inheriting Account and add overdraftLimit.
12. Override withdraw() in CurrentAccount to support the overdraft limit.
13. Create SavingsAccount and CurrentAccount objects.
14. Display details before transactions.
15. Perform deposits, withdrawals, interest calculation, and fund transfer.
16. Display final account details and balances.
17. Stop the program.

4. UML DIAGRAM

Account	
- accountNumber : String	
- accountHolderName : String	
- balance : double	
- accountType : String	
+ Account(...)	
+ deposit(amount) : void	
+ withdraw(amount) : boolean	
+ transfer(to, amount) : boolean	
+ displayAccountDetails() : void	
SavingsAccount	CurrentAccount
- interestRate : double	- overdraftLimit : double
+ calculateInterest() : double	+ withdraw(amount) : boolean [override]

Inheritance relationship: SavingsAccount → Account and CurrentAccount → Account.

5. SOURCE CODE WITH COMMENTS

```
// BankAccountSystem.java
```

```
class Account {
    // Common attributes of every bank account
    protected String accountNumber;
    protected String accountHolderName;
    protected double balance;
    protected String accountType;

    // Constructor to initialize account details
    Account(String accountNumber, String accountHolderName,
            double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Deposit money into the account
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.printf("Deposited Rs. %.2f into %s Account.%n",
                    amount, accountType);
        }
    }

    // Withdraw money when sufficient balance is available
    boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.printf("Withdrew Rs. %.2f from %s Account.%n",
                    amount, accountType);
            return true;
        }
        System.out.println("Insufficient balance.");
    }
}
```

```

        return false;
    }

    // Transfer money from this account to another account
    boolean transfer(Account to, double amount) {
        if (withdraw(amount)) {
            to.deposit(amount);
            return true;
        }
        return false;
    }

    // Display account information
    void displayAccountDetails() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + accountHolderName);
        System.out.println("Account Type : " + accountType);
        System.out.printf("Balance : Rs. %.2f%n", balance);
    }

    double getBalance() {
        return balance;
    }
}

// SavingsAccount inherits from Account
class SavingsAccount extends Account {
    private double interestRate;

    SavingsAccount(String no, String name, double balance,
                    double interestRate) {
        super(no, name, balance, "Savings");
        this.interestRate = interestRate;
    }

    // Calculate interest using balance x rate / 100
    double calculateInterest() {
        double interest = balance * interestRate / 100;
        balance += interest;
        System.out.printf("Interest calculated: Rs. %.2f%n", interest);
        return interest;
    }
}

// CurrentAccount inherits from Account
class CurrentAccount extends Account {
    private double overdraftLimit;

    CurrentAccount(String no, String name, double balance,
                    double overdraftLimit) {
        super(no, name, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Method overriding: allow withdrawal within overdraft limit
    @Override
    boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance + overdraftLimit) {
            balance -= amount;
            System.out.printf("Withdrew Rs. %.2f from Current Account.%n",
                              amount);
            return true;
        }
        System.out.println("Overdraft limit exceeded.");
        return false;
    }
}

public class BankAccountSystem {
    public static void main(String[] args) {

        // Create SavingsAccount and CurrentAccount objects
        SavingsAccount savings =
            new SavingsAccount("SA101", "Biddika", 10000, 1.0);

        CurrentAccount current =
            new CurrentAccount("CA201", "Rukmini", 5000, 5000);

        System.out.println("===== BEFORE TRANSACTIONS =====");
    }
}

```

```

System.out.println("Savings Account");
savings.displayAccountDetails();

System.out.println("\nCurrent Account");
current.displayAccountDetails();

System.out.println("\n----- TRANSACTIONS -----");

// Deposit money into savings account
savings.deposit(2000);

// Withdraw money from savings account
savings.withdraw(1500);

// Calculate and add interest
savings.calculateInterest();

// Transfer money from savings to current account
savings.transfer(current, 2500);

// Current account can use its overdraft facility
current.withdraw(6000);

System.out.println("\n===== FINAL DETAILS =====");
System.out.printf("Savings Account Final Balance : Rs. %.2f%n",
    savings.getBalance());
System.out.printf("Current Account Final Balance : Rs. %.2f%n",
    current.getBalance());
}
}

```

6. OUTPUT SCREENSHOT

The following is a terminal-style output screenshot showing compilation and execution of the Java program.

```
Command Prompt — BankAccountSystem

C:\Java> javac BankAccountSystem.java
C:\Java> java BankAccountSystem

===== BEFORE TRANSACTIONS =====
Savings Account
Account Number : SA101
Account Holder : Biddika
Account Type   : Savings
Balance       : Rs. 10000.00

Current Account
Account Number : CA201
Account Holder : Rukmini
Account Type   : Current
Balance       : Rs. 5000.00

----- TRANSACTIONS -----
Deposited Rs. 2000.00 into Savings Account.
Withdrew Rs. 1500.00 from Savings Account.
Interest calculated: Rs. 105.00
Transferred Rs. 2500.00 from Savings to Current.
Withdrew Rs. 6000.00 from Current Account.
```

7. RESULT

The Bank Account System was successfully designed and implemented in Java. The program demonstrates attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers. The final balances are displayed successfully.